

# ¿Qué es RAG y por qué es el caso de uso #1 en empresa?

## Módulo 7 · Sistemas RAG y Gestión del Conocimiento

---

Retrieval-Augmented Generation: flujo completo, RAG vs fine-tuning, stack de referencia 2025 y el gap prototipo-producción.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales  
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

## ¿Qué es RAG y por qué es el caso de uso #1 en empresa?

RAG es el acrónimo de Retrieval-Augmented Generation: una arquitectura que combina la recuperación de información de bases de conocimiento propias con la capacidad generativa de los LLMs. En diciembre de 2025, la adopción de RAG se aceleró para convertirse en el caso de uso empresarial número uno de LLMs, con el mercado alcanzando 1.85 billones de dólares en 2024 y creciendo al 49% anual. Anthropic, OpenAI y Google recomiendan RAG como la técnica principal para conectar LLMs con datos propietarios empresariales.

El problema que resuelve RAG es fundamental: los LLMs tienen conocimiento general del mundo hasta su fecha de corte, pero no saben nada sobre los documentos internos de tu empresa, las políticas actualizadas, los datos de clientes, o los desarrollos posteriores al entrenamiento. RAG soluciona esto de forma elegante: en lugar de modificar el modelo (fine-tuning), se le proporciona la información relevante directamente en el contexto de cada consulta.

## RAG: libro abierto en lugar de memoria

### La metáfora del libro abierto

La forma más intuitiva de entender RAG es la diferencia entre un examen de memoria cerrado y un examen con libro abierto. Un LLM sin RAG responde de memoria: solo puede usar lo que aprendió durante el preentrenamiento, con todas sus limitaciones (corte de conocimiento, posibles alucinaciones, sin acceso a información privada). Un LLM con RAG responde con libro abierto: antes de generar la respuesta, consulta las fuentes relevantes y construye su respuesta basándose en ellas.

### El flujo básico de RAG

El flujo básico de RAG tiene cuatro etapas. Ingesta: los documentos de la base de conocimiento se dividen en chunks, se generan sus embeddings, y se almacenan en una base de datos vectorial junto con sus metadatos. Recuperación: cuando llega una consulta del usuario, se genera su embedding y se buscan los K chunks más similares en la base vectorial. Aumentación: los chunks recuperados se insertan en el prompt del LLM como contexto adicional. Generación: el LLM genera una respuesta basándose en la consulta y el contexto recuperado.

### RAG vs fine-tuning: cuándo usar cada uno

RAG y fine-tuning responden a necesidades diferentes. RAG es la elección correcta cuando el conocimiento cambia frecuentemente (documentos que se actualizan, datos en tiempo real), cuando la base de conocimiento es grande y diversa, cuando se necesita trazabilidad (¿de qué documento viene esta respuesta?), o cuando los datos son confidenciales y no se quieren incluir en el proceso de entrenamiento. Fine-tuning es mejor cuando se necesita cambiar el estilo, tono o comportamiento del modelo, cuando el dominio es estable y bien definido, o

cuando el rendimiento de RAG no es suficiente para la tarea específica.

### SECCIÓN 3 · CÓMO FUNCIONA

---

## Los componentes técnicos de un sistema RAG

Un sistema RAG completo tiene cinco componentes. El pipeline de ingesta procesa los documentos fuente: parsing (extracción de texto de PDF, Word, HTML), chunking (división en fragmentos manejables), embedding (vectorización de cada chunk), y almacenamiento en la base vectorial con metadatos. La base de datos vectorial almacena embeddings e indexa para búsqueda eficiente por similitud. El sistema de retrieval procesa la consulta del usuario, genera su embedding, y recupera los chunks más relevantes. El prompt builder construye el prompt del LLM combinando la consulta y el contexto recuperado. El LLM generador produce la respuesta fundamentada en el contexto.

En diciembre de 2025, el mercado de bases de datos vectoriales se consolida alrededor de cuatro opciones: Pinecone (SaaS managed, más fácil de empezar), Weaviate (open source + cloud, buen balance), Milvus (open source, mayor escala), y Qdrant (open source, alto rendimiento, fácil de auto-hostear). FAISS de Meta es la librería estándar para búsqueda vectorial en memoria sin necesidad de una base de datos completa.

### SECCIÓN 4 · EJEMPLOS REALES

---

- Harvey AI (legal): RAG sobre millones de documentos de jurisprudencia, contratos y legislación. Cada afirmación incluye cita al documento fuente y fragmento exacto. Sirve al 97% de los bufetes del Am Law 100. Eliminó el problema de las citas alucinadas que llevaron a sanciones judiciales en el caso Mata vs Avianca.
- Morgan Stanley Wealth Management: asistente RAG sobre 100.000 documentos de research financiero para que los asesores encuentren información relevante instantáneamente. El sistema recupera fragmentos relevantes y los cita, permitiendo a los asesores verificar y profundizar en las fuentes.
- Chatbot de soporte técnico de Microsoft (Azure documentación): RAG sobre la documentación técnica completa de Azure. Los usuarios obtienen respuestas precisas a preguntas técnicas complejas con referencias a la sección exacta de la documentación, reduciendo los tickets de soporte repetitivos.
- Asistente de política interna de RRHH (multinacional): RAG sobre el manual del empleado, convenios colectivos, políticas de RRHH actualizadas. Los empleados consultan directamente sin necesidad de abrir tickets de RRHH para preguntas sobre vacaciones, beneficios, o procedimientos de baja.

### SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

---

**Error 1: Construir el RAG antes de definir las preguntas que debe responder.** El diseño del sistema RAG (tamaño de chunk, modelo de embedding, parámetros de recuperación) depende del tipo de preguntas que el sistema debe responder. Empieza definiendo los 20-50 casos de uso más representativos, construye el golden dataset de evaluación, y diseña el sistema para maximizar el rendimiento en esos casos.

Error 2: Asumir que RAG elimina totalmente las alucinaciones. RAG reduce dramáticamente las alucinaciones de conocimiento (el modelo afirma cosas que no están en los documentos) pero no las elimina completamente. El modelo puede aún distorsionar o mal interpretar el contexto recuperado. La evaluación continua con métricas de faithfulness es esencial.

Error 3: Usar el mismo modelo de embedding para todos los tipos de documentos. Los modelos de embedding genéricos tienen peor rendimiento en dominios especializados (médico, legal, técnico). Evalúa el rendimiento de recuperación con tu golden dataset usando varios modelos de embedding antes de construir el índice completo.

## SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso correcto para empezar un proyecto RAG empresarial: primero, define el corpus (qué documentos incluir, con qué criterios de calidad y actualización). Segundo, define las preguntas representativas y construye el golden dataset de evaluación. Tercero, selecciona el modelo de embedding evaluando en el golden dataset. Cuarto, implementa el chunking y la indexación. Quinto, implementa la recuperación y el prompt de generación. Sexto, evalúa con RAGAS antes del despliegue. Séptimo, monitoriza en producción.

El stack tecnológico de referencia para RAG en 2025: LangChain o LlamaIndex como framework de orquestación, Qdrant o Pinecone como base vectorial, Voyage AI o text-embedding-3-large como modelo de embedding, un LLM frontier o modelo fine-tuneado como generador, y RAGAS para evaluación continua.

## SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M2-T6 (Embeddings): los embeddings son el componente central de la recuperación en RAG.
- M7-T2 (Bases vectoriales): el sistema de almacenamiento y búsqueda de los embeddings en RAG.
- M5-T8 (Alucinaciones): RAG es la mitigación más efectiva para alucinaciones de conocimiento.

## SECCIÓN 8 · RESUMEN EJECUTIVO

RAG (Retrieval-Augmented Generation) es la arquitectura que conecta LLMs con bases de conocimiento propietarias mediante recuperación vectorial. Es el caso de uso empresarial número uno en 2025. El flujo es: ingestar documentos → chunking + embedding + indexar en base vectorial → recuperar chunks relevantes por similitud de consulta → insertar en prompt → LLM genera respuesta fundamentada. RAG es mejor que fine-tuning cuando el conocimiento cambia, cuando se necesita trazabilidad, o cuando los datos son confidenciales. El stack de referencia: LangChain/LlamaIndex + Qdrant/Pinecone + Voyage AI/OpenAI embeddings + LLM + RAGAS. El gap prototipo→producción es significativo: requiere gestión del corpus, latencia, multi-tenancy y monitorización.